

Excepciones

Esteban Álvarez

Excepciones

- ¿Qué pasa en este programa si el usuario teclea 0 para divisor?

```
int dividendo=teclado.nextInt();
```

```
int divisor=teclado.nextInt();
```

```
double division=dividendo/divisor;
```

Excepciones

- ¿Qué pasa en este programa si el usuario teclea 0 para divisor?

```
int dividendo=teclado.nextInt();
```

```
int divisor=teclado.nextInt();
```

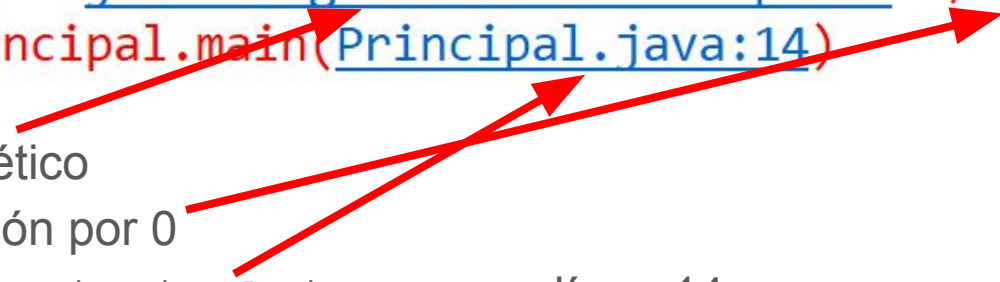
```
double division=dividendo/divisor;
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at Articulos.Principal.main(Principal.java:14)
```

Excepciones

- En determinadas ocasiones, la ejecución de un programa puede generar errores que reciben el nombre de **excepciones**

Exception in thread "main" java.lang.ArithmeticException: / by zero
at Articulos.Principal.main(Principal.java:14)

Three red arrows originate from the list items below and point to specific parts of the exception message above. The first arrow points from 'Excepción de tipo aritmético' to 'ArithmeticException'. The second arrow points from 'Producida por una división por 0' to '/ by zero'. The third arrow points from 'Generada por el fichero Principal.java en su línea 14' to 'Principal.java:14'.

- Excepción de tipo aritmético
- Producida por una división por 0
- Generada por el fichero `Principal.java` en su línea 14

Excepciones

- La sentencia if permite **validar los datos de entrada**
- Modifica el programa anterior para no hacer la división si el usuario teclea 0
para divisor

Excepciones

- La sentencia if permite **validar los datos de entrada**
- Modifica el programa anterior para no hacer la división si el usuario teclea 0 para divisor

```
int dividendo=teclado.nextInt();
int divisor=teclado.nextInt();
if(divisor!=0) {
    double division=dividendo/divisor;
}else {
    System.out.println("El divisor no puede ser 0");
}
```

Excepciones

- Pero a veces **no será posible** hacer una validación
- Ejecuta el programa anterior e introduce "cero" para divisor

```
int dividendo=teclado.nextInt();
int divisor=teclado.nextInt();
if(divisor!=0) {
    double division=dividendo/divisor;
}else {
    System.out.println("El divisor no puede ser 0");
}
```

Excepciones

- Pero a veces **no será posible** hacer una validación
- Ejecuta el programa anterior e introduce "cero" para divisor

```
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:939)
    at java.base/java.util.Scanner.next(Scanner.java:1594)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2258)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2212)
    at Articulos.Principal.main(Principal.java:12)
```

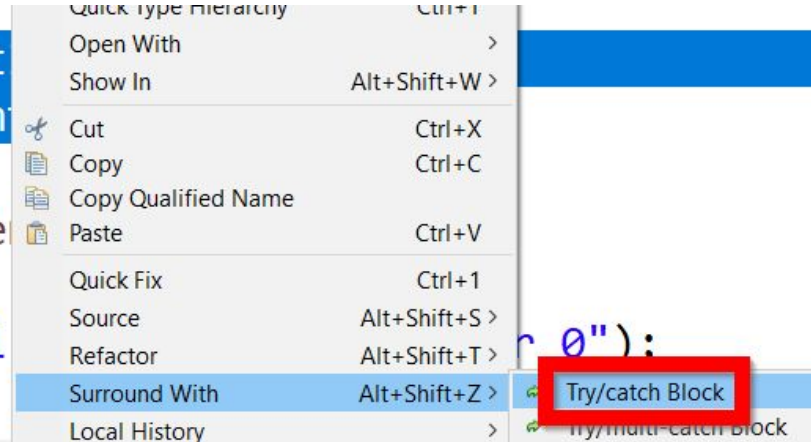
- Se produce una excepción de tipo **Input Mismatch** (tipos incompatibles): se esperaba un entero y se introdujo un String

Excepciones

- Es **imposible** solucionar el problema anterior con un if
- Necesitamos **tratar la excepción**: asumimos que el error puede existir pero actuamos cuando se produzca
- Para ello tenemos el **bloque try-catch**
 - En el bloque **try** ponemos las instrucciones “conflictivas” que pueden generar una excepción
 - Y en el **catch** las acciones que vamos a realizar si se produce la excepción

Excepciones

```
int dividendo=teclado.nextIn  
int divisor=teclado.nextIn  
if(divisor!=0) {  
    double division=divide  
}else {  
    System.out.println("El  
}
```



- Seleccionamos el código conflictivo y rodeamos con un **bloque try/catch**

Excepciones

- Seleccionamos el código conflictivo y rodeamos con un **bloque try/catch**

```
try {  
    dividendo = teclado.nextInt();  
    divisor = teclado.nextInt();  
} catch (Exception e) {  
    // TODO Auto-generated catch block  
    e.printStackTrace();  
}
```

- Por defecto aparece la excepción general `Exception` pero nosotros sabemos que se va a producir una `InputMismatchException` y lo modificamos

```
} catch (InputMismatchException e) {
```

Excepciones

- Ahora **tenemos un ojo** en las dos instrucciones que hay dentro del try
- Si alguna de las dos produce una InputMismatchException, se ejecutará el código que hay dentro del catch

```
try {  
    dividendo = teclado.nextInt();  
    divisor = teclado.nextInt();  
} catch (InputMismatchException e) {  
    // TODO Auto-generated catch block  
    e.printStackTrace();  
}
```

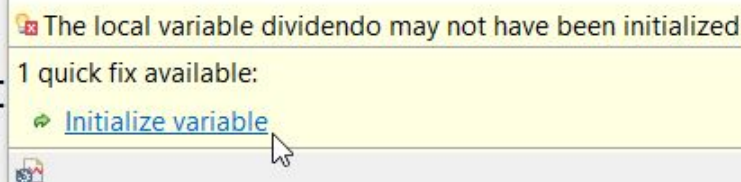
- `e.printStackTrace()` nos muestra ese típico error en rojo
- Lo **modificamos** por algo más “agradable” para el usuario

```
catch (InputMismatchException e) {  
    System.out.println("Sólo números enteros");  
}
```

Excepciones

- El bloque try-catch nos ha generado otro problema: **puede que no se ejecute** el código dentro del try porque se produce una excepción y las variables `dividendo` y `divisor` quedan sin inicializar

```
if(divisor!=0) {  
    double division=dividendo/divisor;  
}else {  
    System.out.print  
}
```



- Las inicializamos a 0, por ejemplo `int dividendo=0;`
`int divisor=0;`

Excepciones

- El tratamiento de las excepciones con try-catch se puede ver como un **IF**:

```
int dividendo = 0;
int divisor = 0;
try {
    dividendo = teclado.nextInt();
    divisor = teclado.nextInt();
} catch (InputMismatchException e) {
    System.out.println("Sólo número");
}

if (divisor != 0) {
    double division = dividendo / divisor;
} else {
    System.out.println("El divisor no puede ser 0");
}
```

El código que hay después del bloque try-catch **se ejecuta siempre**, tanto si se generó una excepción como si no de dentro del catch

Excepciones

- Ahora ejecuta la aplicación y vuelve a escribir "cero" para divisor
- El resultado es el siguiente:

cero

Sólo números enteros 

El divisor no puede ser 0



Excepciones

- Ese mensaje lo genera este bloque de código:

```
int dividendo=0;
int divisor=0;
try {
    dividendo = teclado.nextInt();
    divisor = teclado.nextInt();
} catch (InputMismatchException e) {
    System.out.println("Sólo números enteros");
}
if(divisor!=0) {
    double division=dividendo/divisor;
} else {
    System.out.println("El divisor no puede ser 0");
}
```

- Si el usuario se equivoca al introducir los números no queremos seguir con el programa (validando, calculando la división, etc.)
- Por tanto, **no queremos que se ejecute** ese código

Excepciones

- **No queremos que se ejecute** ese código y para ello lo metemos también dentro del bloque try
- Con eso hacemos que la ejecución sea **atómica**: si se genera una excepción dentro del try, el resto de líneas no se ejecutan

```
try {  
    dividendo = teclado.nextInt();  
    divisor = teclado.nextInt();  
    //Si las dos líneas anteriores producen una excepción  
    //estas líneas siguientes no se ejecutan  
    if(divisor!=0) {  
        double division=dividendo/divisor;  
    }else {  
        System.out.println("El divisor no puede ser 0");  
    }  
} catch (InputMismatchException e) {  
    System.out.println("Sólo números enteros");  
}
```

Excepciones

- Tanto si las instrucciones del bloque try se ejecutan sin problemas como si generan una excepción, **el programa sigue con su ejecución** en la línea siguiente al bloque try-catch:

```
try {  
    ...  
} catch (Exception e) {  
    ...  
}  
System.out.println("Continúa el programa...");
```

¿Excepciones o validación?

- Hay veces que una misma situación se puede resolver **validando con un if** o **tratando la excepción**
- Volvemos al principio. Para evitar una división por 0, validamos el divisor:

```
int dividendo=teclado.nextInt();
int divisor=teclado.nextInt();
if(divisor!=0) {
    double division=dividendo/divisor;
}else {
    System.out.println("El divisor no puede ser 0");
}
```

¿Excepciones o validación?

- Pero podríamos haber **tratado la excepción**

```
try {  
    //Ponemos un try al bloque que puede generar la excepción  
    double division = dividendo / divisor;  
} catch (ArithmeticException e) { //Modificamos el tipo de excepción  
    //Mensaje de error /0  
    System.out.println("No se puede calcular la división");  
    System.out.println("El divisor no puede ser 0");  
}  
System.out.println("Continuamos con otra cosa...");
```

RECOMENDACIÓN: Es preferible validar antes que tratar la excepción

Excepciones

- **Además** del try catch anterior (recuerda, no es la mejor solución), queremos capturar la excepción InputMismatchException
- Para ello **añadimos otro catch al try existente**

```
try {  
    dividendo=teclado.nextInt();  
    divisor=teclado.nextInt();  
    division = dividendo / divisor;  
} catch (ArithmeticException e) {  
    System.out.println("No se puede calcular la divisón");  
    System.out.println("El divisor no puede ser 0");  
} catch (InputMismatchException e) {  
    System.out.println("Sólo números enteros");  
}  
System.out.println("Continuamos con otra cosa...");
```

Conclusión

- El bloque try/catch es **una especie de if**
- Si se **ejecutan correctamente** las instrucciones del try
 - No pasa nada
- Si alguna de las instrucciones del try **genera una excepción**
 - El resto de instrucciones del bloque try no se ejecutan
 - Se ejecuta el código del bloque **catch**
- En cualquier caso, se **continúa** con la ejecución normal del programa

Recuerda poner **dentro del try** todas las instrucciones que quieres que no se ejecuten si alguna de las anteriores genera una excepción

Excepciones y validación

EJERCICIOS

- Hacer las actividades de MENÚ DE OPCIONES del Campus Virtual
- Hacer las actividades de EXCEPCIONES Y VALIDACIÓN del Campus Virtual

FIN!